```
// _worker.js  -  ONE file that does everything, for Cloudflare Pages "drag and d
//   /api/checkout        creates the Stripe payment page (holds the real prices)
//   /api/stripe-webhook  emails each paid order (optional; needs the extra setti
//   everything else      is served from the website files next to this file
// (The separate functions/ folder isn't supported by drag and drop, so both are

const checkoutHandler = (() => {
// functions/api/checkout.js  —  Cloudflare Pages Function for Vintage of 4 Seaso
// It answers at the web address  /api/checkout
//
// The website sends the CART (product names, colors/sizes, quantities) — NEVER p
// This file looks up the real prices below, so a customer can't change what they
// then asks Stripe for a hosted checkout page and sends the customer there.
// No extra libraries needed: it talks to Stripe's web API directly.
//
// Settings (Cloudflare dashboard > your Pages project > Settings > Variables and
//   STRIPE_SECRET_KEY  your Stripe secret key (starts with sk_test_ or sk_live_)
//   SITE_URL           your real site address, starting with https://

// ------------------------------------------------------------------
// PRICE LIST (US dollars). When Luso changes a price, change it here AND on the
// ------------------------------------------------------------------
const CATALOG = {
  "Handcrafted Winter Angel": {"p":84.99},
  "Vintage Carousel Horse Drum Ornament": {"p":74.99},
  "Gold Leaf Teardrop Ornament": {"p":59.99},
  "Antique Carriage Ornament": {"p":124.99},
  "Guitar Ornament": {"p":79.99},
  "Chandelier Ornament": {"p":114.99},
  "High Heel Ornament": {"p":44.99},
  "Medallion Ornament": {"p":94.99},
  "Large Medallion Ornament": {"p":134.99},
  "Mother Mary Ornament": {"p":79.99},
  "Doves & Crown Ornament": {"p":79.99},
  "Throne Ornament": {"p":74.99},
  "Peacock Ornament": {"p":114.99},
  "Ornament Gift Box": {"box":true},
  "Jeweled Bow Ornament": {"p":84.99},
  "Praying Mary Ornament": {"p":79.99},
  "Angel Ornament": {"p":84.99},
  "Sapphire Cameo Ornament": {"p":84.99},
  "Angel of Eternal Love Ornament": {"p":89.99},
  "Ballet Slippers Ornament": {"p":84.99},
```

```
  "Ballerina Ornament": {"p":84.99},
  "Masquerade Mask Ornament": {"p":84.99},
  "Horse Head Ornament": {"p":84.99},
  "Carousel Horse Medallion": {"p":74.99},
};

const has = (obj, key) => Object.prototype.hasOwnProperty.call(obj, key);

// Price for a product + variant. Variants look like "Red" or "Cream · Large";
// for products with sizes, the text after the last " · " picks the price.
function priceFor(name, variant) {
  if (!has(CATALOG, name)) return null;
  const item = CATALOG[name];
  if (item.sizes) {
    const size = String(variant).split(' · ').pop();
    return has(item.sizes, size) ? item.sizes[size] : null;
  }
  return item.p;
}

// Same ZIP-based shipping estimate the website shows. lineCount = number of diff
function shippingCost(lineCount, zip) {
  const base = 6, perExtra = 1.5, extra = (lineCount - 1) * perExtra;
  const digit = zip ? Number(zip[0]) : null;
  let zoneFee = 3;
  if (digit !== null) zoneFee = Math.round(Math.abs(9 - digit) * 0.9 * 10) / 10;
  return Math.round((base + extra + zoneFee) * 100) / 100;
}

const clean = (v, max) => (typeof v === 'string' ? v.replace(/[\u0000-\u001f]/g,

// Gift Box: the price is the ornaments chosen inside it (box included). 3 or mor
const BOX_FEE_CENTS = 0, BOX_FREE_AT = 3, BOX_MAX_ITEMS = 12, BOX_MAX_LINES = 8;
function boxQuote(items) {
  if (!Array.isArray(items) || items.length < 1 || items.length > BOX_MAX_LINES)
  let cents = BOX_FEE_CENTS, count = 0;
  const parts = [];
  for (const it of items) {
    const [n, c, q] = Array.isArray(it) ? it : [];
    const quantity = Number(q);
    if (typeof n !== 'string' || !has(CATALOG, n) || CATALOG[n].box || typeof CAT
    if (!Number.isInteger(quantity) || quantity < 1 || quantity > BOX_MAX_ITEMS)
    count += quantity;
    if (count > BOX_MAX_ITEMS) return null;
    cents += Math.round(CATALOG[n].p * 100) * quantity;
    parts.push(quantity + 'x ' + n.replace(/ Ornament$/, '') + ' (' + clean(typeo
```

```
  }
  return { cents, count, free: count >= BOX_FREE_AT, contents: parts.join(', ') }
}
const say = (message, status) =>
  new Response(message, { status, headers: { 'Content-Type': 'text/plain; charset

async function onRequest(context) {
  const { request, env } = context;

  if (request.method !== 'GET' && request.method !== 'POST') return say('Method n
  if (!env.STRIPE_SECRET_KEY) return say('Server misconfiguration: STRIPE_SECRET_
  const siteUrl = env.SITE_URL;
  if (!siteUrl || !/^https?:\/\//.test(siteUrl)) {
    return say('Server misconfiguration: SITE_URL is missing or must start with h
  }

  // Read the details (address bar for GET, JSON body for POST)
  let q;
  if (request.method === 'GET') {
    const p = new URL(request.url).searchParams;
    q = { cart: p.get('cart'), zip: p.get('zip'), email: p.get('email'), name: p.
  } else {
    try { q = await request.json(); } catch (e) { return say('Invalid request bod
    if (!q || typeof q !== 'object') return say('Invalid request body.', 400);
  }

  let cart;
  try { cart = typeof q.cart === 'string' ? JSON.parse(q.cart) : q.cart; } catch
  if (!Array.isArray(cart) || cart.length < 1 || cart.length > 30) {
    return say('Your cart looks empty. Please go back and try again.', 400);
  }

  // The website sends each line as {priceId, name, color, qty, items?}. The Stri
  // but the price actually charged always comes from CATALOG above (never from t
  const rows = cart.map((r) => (r && typeof r === 'object' && !Array.isArray(r) ?

  // Build the form Stripe expects (every price comes from CATALOG above)
  const form = new URLSearchParams();
  form.append('mode', 'payment');
  let n = 0;
  for (const row of rows) {
    const [name, variant, qty, boxItems] = Array.isArray(row) ? row : [];
    const quantity = Number(qty);
    let unitCents, label;
    if (typeof name === 'string' && has(CATALOG, name) && CATALOG[name].box) {
      const box = boxQuote(boxItems);
```

```
    if (box === null) return say('Your gift box could not be read. Please go ba
    unitCents = box.cents;
    label = clean(name, 60) + ' — ' + clean(String(variant), 30) + ' — contains
    label = label.length > 250 ? label.slice(0, 247) + '...' : label;
  } else {
    const price = typeof name === 'string' ? priceFor(name, variant) : null;
    if (typeof price !== 'number') return say('One of the items in your cart co
    unitCents = Math.round(price * 100);
    label = clean(name, 100) + ' — ' + clean(String(variant), 60);
  }
  if (!Number.isInteger(quantity) || quantity < 1 || quantity > 50) {
    return say('One of the items in your cart could not be found. Please go bac
  }
  form.append(`line_items[${n}][price_data][currency]`, 'usd');
  form.append(`line_items[${n}][price_data][product_data][name]`, label);
  form.append(`line_items[${n}][price_data][unit_amount]`, String(unitCents));
  form.append(`line_items[${n}][quantity]`, String(quantity));
  n++;
}

const zip = /^\d{5}$/.test(clean(q.zip, 5)) ? clean(q.zip, 5) : '';
const ship = shippingCost(cart.length, zip);
form.append(`line_items[${n}][price_data][currency]`, 'usd');
form.append(`line_items[${n}][price_data][product_data][name]`, 'Shipping (esti
form.append(`line_items[${n}][price_data][unit_amount]`, String(Math.round(ship
form.append(`line_items[${n}][quantity]`, '1');

const email = clean(q.email, 200);
if (/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)) form.append('customer_email', ema
form.append('shipping_address_collection[allowed_countries][0]', 'US');
form.append('phone_number_collection[enabled]', 'true');
form.append('metadata[name]', clean(q.name, 200));
form.append('metadata[phone]', clean(q.phone, 40));
form.append('metadata[address]', clean(q.addr, 300));
form.append('metadata[zip]', zip);
form.append('success_url', siteUrl + (siteUrl.includes('?') ? '&' : '?') + 'pai
form.append('cancel_url', siteUrl);

// Ask Stripe for the hosted checkout page
let data = {};
let ok = false;
try {
  const resp = await fetch('https://api.stripe.com/v1/checkout/sessions', {
    method: 'POST',
    headers: { Authorization: 'Bearer ' + env.STRIPE_SECRET_KEY, 'Content-Type'
    body: form.toString(),
```

```
    });
    data = await resp.json().catch(() => ({}));
    ok = resp.ok && !!data.url;
    if (!ok) console.error('Stripe error', resp.status, data && data.error && dat
  } catch (e) {
    console.error('Could not reach Stripe', e && e.message);
  }
  if (!ok) return say('Something went wrong starting checkout. Please go back and

  if (request.method === 'POST') {
    return new Response(JSON.stringify({ url: data.url }), { status: 200, headers
  }
  return new Response(null, { status: 303, headers: { Location: data.url } });
}

  return onRequest;
})();

const webhookHandler = (() => {
// functions/api/stripe-webhook.js  —  Cloudflare Pages Function
// It answers at the web address  /api/stripe-webhook
//
// What it does: Stripe calls this address after every successful payment.
// We check the call really came from Stripe, then email the order details
// (items, total, customer, shipping address) to the shop's email address.
//
// Settings (Cloudflare dashboard > your Pages project > Settings > Variables and
//   STRIPE_SECRET_KEY       your Stripe secret key (same one the checkout uses)
//   STRIPE_WEBHOOK_SECRET   the "Signing secret" Stripe shows for this webhook (
//   RESEND_API_KEY          your Resend API key (starts with re_)
//   ORDER_EMAIL             where order emails go, for example lisaabrahamian38@
//   ORDER_FROM              (optional) the "from" line, once you have a verified
//                           for example  Vintage of 4 Seasons <orders@yourdomain

const say = (message, status) =>
  new Response(message, { status, headers: { 'Content-Type': 'text/plain; charset

const money = (cents) => '$' + (Number(cents || 0) / 100).toFixed(2);

// ---- check the call is really from Stripe (signature check, no libraries neede
const toHex = (buf) => [...new Uint8Array(buf)].map((b) => b.toString(16).padStar

async function hmacHex(secret, message) {
  const key = await crypto.subtle.importKey('raw', new TextEncoder().encode(secre
  return toHex(await crypto.subtle.sign('HMAC', key, new TextEncoder().encode(mes
}
```

```
function sameText(a, b) {
  if (a.length !== b.length) return false;
  let diff = 0;
  for (let i = 0; i < a.length; i++) diff |= a.charCodeAt(i) ^ b.charCodeAt(i);
  return diff === 0;
}

async function isFromStripe(rawBody, header, secret, nowSeconds = Date.now() / 10
  if (!header) return false;
  let timestamp = '';
  const signatures = [];
  for (const piece of header.split(',')) {
    const [k, v] = piece.split('=');
    if (k === 't') timestamp = v;
    if (k === 'v1' && v) signatures.push(v);
  }
  if (!timestamp || signatures.length === 0) return false;
  if (Math.abs(nowSeconds - Number(timestamp)) > 300) return false; // older than
  const expected = await hmacHex(secret, timestamp + '.' + rawBody);
  return signatures.some((s) => sameText(s, expected));
}

// ---- write the email (plain text on purpose: nothing a customer typed can run
function buildEmail(session) {
  const customer = session.customer_details || {};
  const collected = (session.collected_information && session.collected_informati
  const a = collected.address || customer.address || {};
  const shipTo = [
    collected.name || customer.name,
    a.line1,
    a.line2,
    [a.city, a.state, a.postal_code].filter(Boolean).join(' '),
    a.country,
  ].filter(Boolean).join('\n');
  const lines = ((session.line_items && session.line_items.data) || [])
    .map((li) => '- ' + li.quantity + ' x ' + li.description + '   ' + money(li.a
    .join('\n');
  const meta = session.metadata || {};
  const who = customer.name || meta.name || 'customer';
  const payment = session.payment_intent
    ? 'https://dashboard.stripe.com/payments/' + session.payment_intent
    : 'https://dashboard.stripe.com/payments';
  const text = [
    'NEW PAID ORDER',
    '',
```

```
    'Customer: ' + who,
    'Email: ' + (customer.email || '(none)'),
    'Phone: ' + (customer.phone || meta.phone || '(none)'),
    '',
    'SHIP TO',
    shipTo || '(no address collected)',
    '',
    'ITEMS',
    lines || '(could not read items - check Stripe)',
    '',
    'TOTAL PAID: ' + money(session.amount_total),
    '',
    meta.address ? 'Address typed on the website: ' + meta.address : '',
    'See it in Stripe: ' + payment,
  ].filter((l, i, all) => !(l === '' && all[i - 1] === '')).join('\n');
  return { subject: 'New order ' + money(session.amount_total) + ' - ' + who, tex
}

async function onRequest(context) {
  const { request, env } = context;
  if (request.method !== 'POST') return say('Method not allowed', 405);
  for (const name of ['STRIPE_SECRET_KEY', 'STRIPE_WEBHOOK_SECRET', 'RESEND_API_K
    if (!env[name]) return say('Server misconfiguration: ' + name + ' is missing.
  }

  const raw = await request.text();
  if (!(await isFromStripe(raw, request.headers.get('Stripe-Signature'), env.STRI
    return say('Bad signature', 400);
  }
  let event;
  try { event = JSON.parse(raw); } catch (e) { return say('Bad body', 400); }

  // We only care about finished payments; anything else is fine to ignore.
  if (event.type !== 'checkout.session.completed') return say('Ignored', 200);
  const done = event.data && event.data.object;
  if (!done || !done.id) return say('No session', 400);
  if (done.payment_status !== 'paid') return say('Not paid yet', 200);

  // Ask Stripe for the full order (the event itself does not list the items)
  const got = await fetch('https://api.stripe.com/v1/checkout/sessions/' + encode
    headers: { Authorization: 'Bearer ' + env.STRIPE_SECRET_KEY },
  }).catch(() => null);
  if (!got || !got.ok) { console.error('Could not read the order from Stripe'); r
  const session = await got.json();

  const { subject, text } = buildEmail(session);
```

```
  const sent = await fetch('https://api.resend.com/emails', {
    method: 'POST',
    headers: { Authorization: 'Bearer ' + env.RESEND_API_KEY, 'Content-Type': 'ap
    body: JSON.stringify({
      from: env.ORDER_FROM || 'Vintage of 4 Seasons <onboarding@resend.dev>',
      to: [env.ORDER_EMAIL],
      subject,
      text,
    }),
  }).catch(() => null);
  // If the email fails we answer 500 so Stripe automatically tries again later.
  if (!sent || !sent.ok) { console.error('Order email failed'); return say('Email
  return say('ok', 200);
}

  return onRequest;
})();

export default {
  async fetch(request, env) {
    const { pathname } = new URL(request.url);
    if (pathname === '/api/checkout') return checkoutHandler({ request, env });
    if (pathname === '/api/stripe-webhook') return webhookHandler({ request, env
    return env.ASSETS.fetch(request);
  },
};
```